

CSCE 763: Digital Image Processing Homework #5

Instructor: Spring 2026
University of South Carolina

Due: 1:15 pm EST, Thursday, April 9

Table of Contents	
Problem 1: Image Entropy and Compression (25 pts)	2
Problem 2: Arithmetic Decoding (25 pts)	3
Problem 3: Line Break Detection Masks (25 pts)	4
Problem 4: Sobel Mask Decomposition (25 pts)	5

Problem 1: Image Entropy and Compression (25 pts)

Consider a simple 4×16 , 8-bit image:

22	22	21	94	93	94	166	167	166	167	253	254	254	255	253	55
20	22	21	22	94	95	93	166	167	166	254	253	255	255	254	54
21	21	20	22	95	95	95	165	166	166	255	254	254	254	253	54
21	21	22	95	95	94	166	167	165	166	254	255	253	253	255	55

- (a) Compute the entropy of the image. (5 pts)
- (b) Compress the image using Huffman coding and report the compression ratio using Huffman coding. (10 pts)
- (c) Use another data compression algorithm shown in the lecture to obtain a higher compression ratio. (10 pts)

Problem 2: Arithmetic Decoding (25 pts)

The arithmetic decoding process is the reverse of the encoding procedure. Decode the message 0.222355 given the coding model. Assume that “!” is the symbol indicating the end of message. Show the decoding process.

Symbol	Probability
A	0.25
B	0.15
C	0.1
D	0.3
E	0.1
!	0.1

Problem 3: Line Break Detection Masks (25 pts)

A binary image contains straight lines oriented horizontally, vertically, at 45° , and at -45° . Develop a set of 3×3 masks that can be used to detect 1-pixel breaks in these lines. For example, you may want to detect a pattern like 1 1 1 0 1 1 1 in the vertical direction. Assume that the intensities of the lines and background are represented by 1 and 0, respectively.

Problem 4: Sobel Mask Decomposition (25 pts)

The results obtained by a single pass through an image of some 2D masks can be achieved also by two passes using 1D masks for improving efficiency. Show that the response of Sobel masks

$$\mathbf{S}_x = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (1)$$

$$\mathbf{S}_y = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (2)$$

can be implemented similarly by one pass of a vertical differencing mask

$$\mathbf{d}_v = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \quad (3)$$

followed by a horizontal smoothing mask

$$\mathbf{s}_h = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \quad (4)$$